

BASES DE DATOS HIGH LOAD

PostgreSQL & Redis: Optimización a 10,000 QPS

Domina el particionamiento de tablas, indexación avanzada B-Tree/GIN, réplicas de lectura, clustering Redis y resolución de cuellos de botella de alto tráfico.

[Nivel: Avanzado]

[Duración: 14hs]

[10 Módulos]

[Tech: PostgreSQL, Redis, Docker]

#> Módulo 1: Fundamentos de Alto Rendimiento

El desarrollo de aplicaciones modernas requiere bases de datos capaces de soportar miles de consultas por segundo (QPS). La diferencia entre un desarrollador Junior y un Arquitecto de Base de Datos radica en entender qué sucede en el hardware cuando se ejecuta una consulta SQL.

El Mito de las 10,000 QPS

Alcanzar 10k QPS no se trata de usar una 'base de datos más rápida', sino de eliminar cuellos de botella de I/O (lectura/escritura en disco) y aprovechar la memoria RAM. PostgreSQL puede superar este límite si sabemos cómo estructurar los datos y manejar la concurrencia (ACID).

#> Módulo 2: Internals de PostgreSQL (MVCC & WAL)

Para optimizar PostgreSQL, debemos entender cómo procesa los datos internamente.

MVCC (Multi-Version Concurrency Control)

Cuando haces un UPDATE, Postgres NO borra la fila antigua. En su lugar, crea una nueva versión de la fila. Esto permite que otras transacciones sigan leyendo la fila vieja sin ser bloqueadas. Sin embargo, esto genera 'Dead Tuples' (basura).

Write-Ahead Logging (WAL)

Para ser rápido y seguro frente a apagones, Postgres escribe todos los cambios en un archivo secuencial llamado WAL antes de escribir en el archivo de datos real.

```
-- Ver estadísticas de tuplas muertas que necesitan Vacuum:  
SELECT relname, n_dead_tup  
FROM pg_stat_user_tables  
ORDER BY n_dead_tup DESC;
```

El proceso de AUTOVACUUM es crucial: limpia las tuplas muertas. Jamás desactives el autovacuum en producción.

#> Módulo 3: Indexación Avanzada B-Tree

Un índice no es magia; es una estructura de datos (generalmente un Árbol Balanceado - B-Tree) que guarda punteros a la ubicación física de las filas en el disco.

Índices Compuestos y Orden

Si creas un índice en (A, B), el índice solo será útil si tu consulta busca por A, o por A y B. Si buscas solo por B, el índice compuesto NO sirve (regla del prefijo más a la izquierda).

```
-- Crear un índice compuesto:
CREATE INDEX idx_user_status_date ON users (status, created_at DESC);

-- Esta consulta usará el índice perfectamente:
SELECT * FROM users WHERE status = 'active' ORDER BY created_at DESC;
```

Index-Only Scans y Fillfactor

Si tu consulta SELECT solo pide columnas que YA están en el índice, Postgres no necesita leer la tabla en el disco. A esto se le llama Index-Only Scan y es el Santo Grial del rendimiento de lectura.

#> Módulo 4: Índices Especiales (GIN & GiST)

El B-Tree es excelente para igualdades y rangos (>, <). Pero falla para búsquedas dentro de arreglos, documentos JSON, o texto libre (Full-Text Search).

El Índice GIN (Generalized Inverted Index)

GIN es ideal para columnas JSONB. Construye un 'índice invertido', mapeando cada llave/valor del JSON a las filas que lo contienen.

```
-- Suponiendo una columna JSONB llamada 'metadata'
CREATE INDEX idx_gin_metadata ON products USING GIN (metadata);

-- Búsqueda ultra rápida dentro de un JSON gigante:
SELECT * FROM products WHERE metadata @> '{"color": "red"}';
```

Sin GIN, Postgres tendría que hacer un 'Seq Scan' (leer toda la tabla) y abrir cada JSON uno por uno, destruyendo el rendimiento.

#> Módulo 5: Query Tuning y EXPLAIN ANALYZE

Nunca intentes adivinar por qué una consulta es lenta. Usa el profiler interno de PostgreSQL.

Anatomía del EXPLAIN ANALYZE

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 15;
```

Al ejecutar esto, Postgres te dirá:

- Seq Scan: Leyó toda la tabla (¡Peligro si es grande!).
- Index Scan: Usó el índice y luego fue a la tabla a buscar los datos.
- Execution Time: El tiempo real que tomó en milisegundos.
- Buffers: Cuántos bloques de memoria tuvo que tocar.

Si ves un 'Seq Scan' en una tabla enorme, significa que necesitas un índice, o que la condición de búsqueda está ocultando el índice (ej. `WHERE LOWER(email) = ...` requiere un índice funcional).

#> Módulo 6: Particionamiento de Tablas

Cuando una tabla supera las decenas de millones de filas, incluso los índices se vuelven lentos porque el árbol B-Tree es demasiado profundo. La solución es dividir físicamente la tabla.

Particionamiento Declarativo por Rango

```
-- 1. Creamos la tabla 'padre' definiendo el tipo de partición
CREATE TABLE sales (
    id serial,
    amount numeric,
    created_at date
) PARTITION BY RANGE (created_at);

-- 2. Creamos las particiones 'hijas'
CREATE TABLE sales_2024_q1 PARTITION OF sales
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');

CREATE TABLE sales_2024_q2 PARTITION OF sales
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

Cuando haces un SELECT para Marzo 2024, Postgres ignora por completo la tabla `sales\_2024\_q2`. Has reducido el volumen de búsqueda a la mitad instantáneamente.

#> Módulo 7: Replicación y Pooling (PgBouncer)

Ningún servidor físico puede soportar infinitas lecturas. Para escalar horizontalmente, creamos réplicas.

Read Replicas

A través de 'Streaming Replication', Postgres copia los registros WAL de la base de datos principal (Master/Primary) a un servidor esclavo (Replica) en tiempo real. Tu aplicación puede enviar todos los SELECT (lecturas) a la réplica, liberando a la base de datos principal para que se concentre solo en INSERTs y UPDATEs.

Connection Pooling con PgBouncer

Postgres consume RAM (unos 10MB) por cada conexión abierta. Si tu aplicación abre 5000 conexiones, Postgres colapsará. PgBouncer se coloca en medio: recibe las 5000 conexiones de la app, y las reutiliza sobre 100 conexiones reales a Postgres.

#> Módulo 8: Arquitectura In-Memory (Redis)

Incluso con índices y réplicas, leer de un disco SSD toma milisegundos. Para alcanzar latencias de microsegundos, debemos saltarnos el disco y leer directo de la memoria RAM. Aquí entra Redis.

Estructuras de Datos Nativas

Redis no guarda tablas, guarda estructuras de datos: Strings, Hashes, Lists, Sets y Sorted Sets.

```
# Usando redis-cli:
# Guardar una sesión en un Hash
HSET session:123 user_id 45 name "Neo"

# Recuperar el nombre en O(1) (microsegundos)
HGET session:123 name
```

#> Módulo 9: Estrategias de Caché Extrema

¿Cuándo escribimos en Redis y cuándo en Postgres?

Patrón Cache-Aside

1. La app pide datos (ej. perfil del usuario).
2. Primero busca en Redis.
3. Si hay 'Cache Hit', lo devuelve de inmediato.
4. Si hay 'Cache Miss', va a Postgres, procesa, lo devuelve y LO GUARDA en Redis para la próxima vez, asignándole un TTL (Time-To-Live).

```
import redis, psycopg2

r = redis.Redis(host='localhost', port=6379, db=0)

def get_user(user_id):
    # Intentar caché primero
    user_data = r.get(f"user:{user_id}")
    if user_data:
        return user_data

    # Si falla, ir a Postgres
    # db_data = run_sql(f"SELECT * FROM users WHERE id={user_id}")

    # Guardar en Redis con expiración de 10 minutos (600s)
    r.setex(f"user:{user_id}", 600, db_data)
    return db_data
```

Con esto proteges a Postgres de consultas repetitivas masivas (Ej: conteo de likes en un post viral).

#> Módulo 10: Redis Clustering y Sharding

Si tienes demasiada caché, llenas la RAM de un solo servidor. Redis Cluster permite distribuir las llaves (Sharding) a lo largo de docenas de servidores Redis automáticamente.

Resolución de 'Hot Keys'

Si un influencer publica un video, la llave `video:123:likes` recibirá millones de requests, saturando el servidor que posee esa llave. La solución es agregar caché a nivel local en la aplicación (L1 Cache) antes de consultar a Redis (L2 Cache), mitigando el efecto de avalancha.

¡Felicidades! Has completado el entrenamiento de High Load. Estás listo para diseñar arquitecturas de bases de datos que soportan millones de usuarios activos diarios (DAU).

HIGH-PERFORMANCE DATABASE ENGINEER

Certificación otorgada a:

[DATABASE ARCHITECT]

Por haber dominado el diseño y escalado de bases de datos relacionales y en memoria,
resolviendo cuellos de botella de concurrencia y alcanzando un alto throughput de I/O
en entornos de producción masivos.

TIMESTAMP: 2026-08-12 // SYS_DBA_SIGNATURE